[NAME OF THE INSTITUTE]

Department of Computer Science and Engineering

B.TECH FINAL YEAR PROJECT REPORT

SecureX: Cybersecurity Threat Advisor

An explainable threat detection system with risk-based, human-approved response for Windows computers



Submitted by	
Roll number(s)	
Under the guidance of	
Domain	Cybersecurity and Artificial Intelligence
Software version	SecureX 1.2.0

Academic Year 2026–27

Abstract

Ordinary computer users and small organisations often cannot tell whether their computer is under attack. Traditional antivirus software either blocks things silently or shows technical warnings that are hard to understand, and it rarely explains *why* something is dangerous or *what* to do next.

This project presents **SecureX**, a threat detection and advisory system for Windows computers. SecureX continuously watches running programs, network connections and files in important folders. It detects suspicious behaviour using three complementary methods: (i) a set of expert-written detection rules, (ii) a behaviour-based ransomware detector, and (iii) a machine-learning model (Isolation Forest) that learns the normal behaviour of the machine and flags unusual activity. Each finding is given a risk score that also considers context such as how important the device is and whether the activity happened at night. SecureX then explains the threat in plain language, and for serious threats it *proposes* a containment action, such as pausing the suspicious program, which runs only after a human approves it. Every important action is written to a tamper-evident audit log.

The system is delivered as a web dashboard and a native Windows desktop application, and includes an AI security assistant that answers questions in simple language. A separate, harmless *Attack Tester* tool was built to demonstrate the system: it imitates five kinds of attack, and SecureX detected every one in testing. The software is covered by 212 automated tests.

Keywords: intrusion detection, anomaly detection, Isolation Forest, ransomware detection, risk assessment, explainable security, human-in-the-loop response, audit logging.

Table of Contents

1 Introduction 4

1.1 Background · 1.2 Problem statement · 1.3 Objectives · 1.4 Scope

2 Literature Review and Research Gaps 6

3 Proposed System 7

3.1 Overview · 3.2 Key features · 3.3 Technology stack

4 Design Workflow 9

4.1 Development methodology · 4.2 End-to-end workflow · 4.3 User workflow · 4.4 Design principles

5 High-Level Design (HLD) 11

5.1 System architecture · 5.2 Components · 5.3 Deployment view · 5.4 Data flow diagram

6 Low-Level Design (LLD) 14

6.1 Modules · 6.2 Scan pipeline · 6.3 Detection logic · 6.4 Risk scoring

6.5 Response state machine · 6.6 Database design · 6.7 API design · 6.8 Security design

7 Implementation and User Interface 20

8 Testing and Results 22

9 Limitations and Future Work 23

10 Conclusion 24

References 25

List of Figures

Figure 4.1 End-to-end detection and response workflow 9

Figure 4.2 User workflow for testing and responding to a threat 10

Figure 5.1 Layered system architecture 11

Figure 5.2 Deployment view on a single Windows computer 12

Figure 5.3 Context-level data flow diagram (DFD Level 0) 13

Figure 6.1 Scan pipeline (one monitoring cycle) 15

Figure 6.2 Life cycle of a response proposal 17

Figure 6.3 Core database relationships 18

Figure 7.1 Public landing page · Figure 7.2 Security dashboard 20

CHAPTER 1

Introduction

1.1 Background

Cyber attacks are no longer aimed only at large companies. Students, home users and small offices are regularly hit by malware, ransomware (programs that lock files and demand money), phishing links and remote-access backdoors. Most of these people are not security experts. When a security product shows an alert such as `Trojan:Win32/Generic`, the user usually does not know how serious it is or what to do about it.

Research in intrusion detection has produced many accurate detection techniques, including rule-based systems and machine-learning models. However, much of this work stops at *detection*. It pays less attention to explaining results to people, judging risk in context, and responding safely and accountably. SecureX was designed to close these gaps in a practical, working system.

1.2 Problem Statement

> To design and build a system that detects suspicious activity on a Windows computer, judges how dangerous it is in context, explains it in plain language, and helps the user respond safely, while keeping a human in control of every containment action and keeping a trustworthy record of what happened.

1.3 Objectives

1. **Monitor** running processes, network connections and important folders in real time.
2. **Detect** threats using a hybrid approach: expert rules, behaviour-based ransomware detection and machine-learning anomaly detection.
3. **Assess risk** with a transparent, weighted score that also considers device importance, network exposure, privileged accounts and time of day.
4. **Explain** every threat in simple language and offer an AI assistant for questions.
5. **Respond safely** by proposing actions (pause or stop a program) that run only after human approval.
6. **Be accountable** by recording all security-relevant actions in a tamper-evident audit log.
7. **Be usable** through a modern web dashboard and a one-click Windows desktop application.

1.4 Scope

SecureX protects a single Windows computer and is presented as a **research prototype**. It is not intended to replace commercial antivirus software. It does not scan file contents for virus signatures.

Instead it focuses on *behaviour*: what programs are doing, whom they are talking to, and how files are being changed.

CHAPTER 2

Literature Review and Research Gaps

Surveys of machine-learning methods for intrusion detection [1] show that anomaly-based methods can find previously unseen attacks, but they also show that results depend heavily on the dataset and evaluation method. Deep-learning approaches [2] and online ensembles such as Kitsune [3] report high accuracy on benchmark datasets such as CIC-IDS2017 [4] and UNSW-NB15 [5]. The Isolation Forest algorithm [6] is well suited to anomaly detection because it needs only normal data to train and is fast enough to run continuously.

From the reviewed literature, seven research gaps were identified. Table 2.1 lists each gap in simple terms and how SecureX addresses it.

Research gap	The problem in simple words	How SecureX addresses it
G1. Evaluation validity	Accuracy figures are often inflated by duplicate data or unfair train/test splits.	A careful evaluation pipeline for CIC-IDS2017 and UNSW-NB15 that removes duplicates, trains only on normal traffic and reports precision, recall, F1 and false-positive rate.
G2. Detection without response	Systems raise alarms but do not help users act.	Every threat receives an advisory and a response level (observe, alert, contain, terminate). The outcome of every response is recorded.
G3. Weak adaptivity	Fixed rules cannot recognise new kinds of attack.	Continuous monitoring plus an Isolation Forest that learns this machine's normal behaviour and flags unknown suspicious activity.
G4. Missing risk context	The same event is treated equally on every machine and at every time.	The risk score is adjusted for device importance, internet exposure, privileged accounts and off-hours activity. The reasons are shown to the user.
G5. Poor explainability	Alerts are technical and do not support decisions.	Plain-language advisories and an AI assistant that separates evidence from recommendations and warns when it is uncertain.
G6. Unsafe automatic response	Fully automatic blocking can be abused or can break the system.	Containment is only *proposed*. It needs human approval, protects system processes, expires after 30 minutes, is rate-limited and can be undone.
G7. Accountability	It is unclear who did what, and records can be altered.	A hash-chained audit log of logins, configuration changes and every response decision, with an integrity check.

Table 2.1 *Research gaps and the corresponding SecureX features*

CHAPTER 3

Proposed System

3.1 Overview

SecureX works like a security guard for the computer. It **watches** what is happening, **recognises** suspicious behaviour, **decides** how serious it is, **explains** it to the user and **suggests** what to do. Dangerous actions are never taken automatically: the user stays in charge.

3.2 Key Features

Feature	What it does
Real-time monitoring	Checks running programs, CPU and memory use, and network connections every few seconds, and watches the Documents, Desktop and Pictures folders for file changes.
Rule-based detection	Ten expert rules recognise known bad signs, for example hacking-tool names, connections to backdoor ports, or a program opening an unusually large number of connections.
Ransomware detection	Spots “encryption-like” behaviour: many files renamed in a short time, files changed to strange extensions such as `.locked`, or a ransom note appearing. It also identifies which program is most likely responsible.
Unknown-threat detection (ML)	Learns the normal behaviour of each program on this machine using an Isolation Forest, then reports programs that behave very differently.
Context-aware risk scoring	Combines severity, confidence, impact and frequency into a 0–1 score, then adjusts it for context. The result is labelled Low, Medium, High or Critical.
Plain-language advisories	Explains what the threat is, why it matters and what steps to take.
Response Center	For High and Critical threats linked to a running program, proposes “suspend” (pause, reversible) or “terminate”. Nothing runs until a signed-in user approves.
AI security assistant	Answers questions such as “What is phishing?” or “Why was this flagged?” using a local knowledge base and the user’s own threat records. It works offline by default.
Email alerts	Sends alerts for serious threats to verified email addresses, without sending repeated emails for the same ongoing threat.
Audit log	Records who did what and when, in a chain of hashes so that tampering can be detected.
User accounts	Sign-up and sign-in with securely hashed passwords. Each user’s assistant history is private, and users can delete it.

Two interfaces	A web dashboard in any browser, and a desktop application (a Windows `.exe`) that embeds the same dashboard.

Table 3.1 *Main features of SecureX*

3.3 Technology Stack

Layer	Technology	Why it was chosen
Backend language	Python 3.12	Rich libraries for systems, security and machine learning.
Web server and API	Flask, Flask-SocketIO	Lightweight REST API with live updates pushed to the browser.
System monitoring	psutil, watchdog	Read processes and connections, and receive file-change events.
Machine learning	scikit-learn (Isolation Forest), NumPy	Fast, unsupervised anomaly detection that trains on normal data only.
AI assistant	Local knowledge base; optional LangChain, Sentence-Transformers and ChromaDB	Retrieval-augmented answers. Works offline without an API key.
Database	SQLite	Zero setup, a single file, suitable for one machine.
Frontend	React 19, TypeScript, Vite, Tailwind CSS, TanStack Query, Recharts	Modern, type-safe and responsive user interface with charts.
Desktop application	PyQt6 with QtWebEngine, packaged by PyInstaller	Runs the server and dashboard inside one native Windows window.
Testing	pytest	212 automated tests across 20 test files.
Deployment	Windows `.exe`; Docker (optional)	One-click use on Windows; container option for servers.

Table 3.2 *Technology stack*

CHAPTER 4

Design Workflow

4.1 Development Methodology

The project followed an **iterative and incremental** model. Each iteration added one capability and was tested before the next began:

1. **Requirements and research:** literature review, identification of the seven research gaps, choice of datasets.
2. **Core detection:** system and network monitors, rule engine, SQLite storage, command-line interface.
3. **Risk and advice:** weighted risk scorer, context adjustment, plain-language advisor.
4. **Machine learning:** Isolation Forest behaviour baseline and the dataset evaluation pipeline.
5. **Interface:** Flask REST API, React dashboard, live updates, desktop application.
6. **Governance:** graduated response with human approval, audit log, user accounts, email alerts.
7. **Assistant and validation:** AI assistant, the Attack Tester and automated tests.

4.2 End-to-End Workflow

Figure 4.1 shows what happens from the moment something suspicious occurs on the computer until the user sees it and responds. The dashed box marks the step where a human must decide.

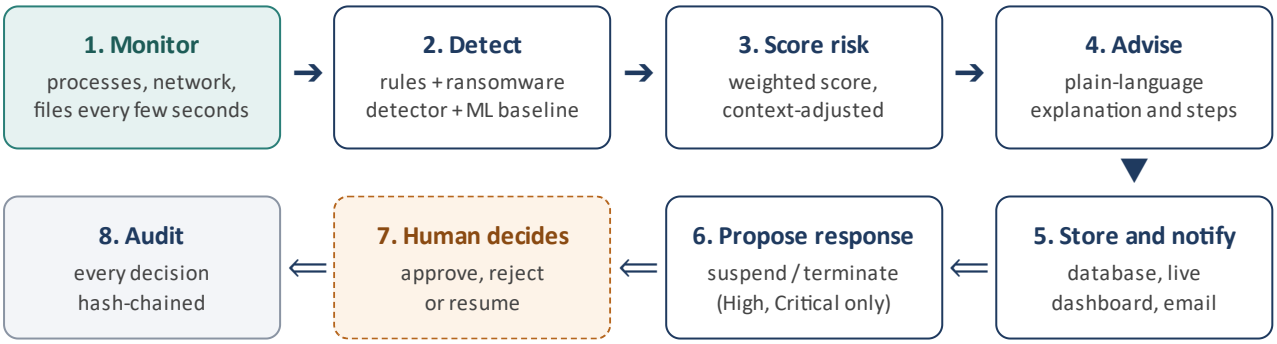


Figure 4.1 *End-to-end detection and response workflow*

In words: monitors collect *events*; detectors turn suspicious events into *threats*; each threat gets a *risk score* and an *advisory*; it is saved and pushed to the dashboard; if it is High or Critical and linked to a running program, a *response proposal* is created; the user approves or rejects it; and every step is written to the audit log.

4.3 User Workflow

Figure 4.2 shows how a user (or an examiner during a demonstration) interacts with SecureX.

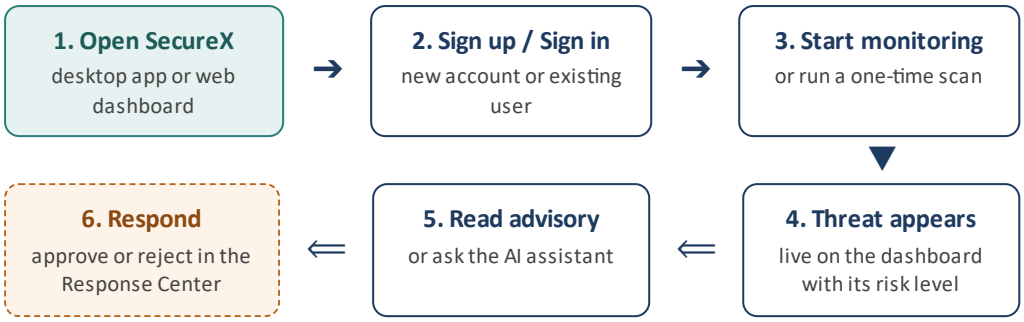


Figure 4.2 *User workflow for testing and responding to a threat*

4.4 Design Principles

The following principles guided every design decision in SecureX:

Principle	Meaning in SecureX
Human in the loop	The system may detect and recommend, but only a person may approve an action that affects a running program.
Explainability	Every threat carries its evidence, the reasons for its risk level and a plain-language advisory.
Defence in depth	Three independent detectors (rules, ransomware behaviour, ML baseline) cover each other's blind spots.
Fail-safe defaults	Response execution is off by default; actions expire if nobody decides; system processes are protected.
Least privilege	The AI assistant has no tools and cannot act; each user sees only their own assistant history.
Privacy by design	Only file paths and extensions are recorded, never file contents; everything runs locally by default.
Accountability	Every security-relevant action is written to a tamper-evident, hash-chained audit log.
Modularity	Each layer and component has one responsibility and can be tested on its own.

Table 4.1 *Design principles*

CHAPTER 5

High-Level Design (HLD)

The high-level design describes the main building blocks of the system and how they connect, without going into code details.

5.1 System Architecture

SecureX uses a **layered architecture**. Each layer only talks to the layer directly below it, which keeps the parts independent and easy to test.



Figure 5.1 *Layered system architecture*

5.2 Major Components

Component	Responsibility
Monitors	Collect raw facts: running processes and their resource use, active network connections, and file events in watched folders.
Detection engine	Turns raw facts into candidate threats using rules, ransomware behaviour signals and the ML behaviour baseline.
Risk scorer	Calculates a 0–1 risk score and a level (Low to Critical), and records the reasons for any context adjustment.
Security advisor	Writes a short, non-technical explanation and recommended steps for each threat.
Response manager	Creates containment proposals according to policy, checks them again at approval time, executes them (or dry-runs them) and supports undo.
AI assistant	Answers user questions using retrieved documents and the user's threat records. It has no ability to run commands.
Notification manager	Sends email alerts for serious threats on a background queue, with a cooldown that prevents repeated emails.
Audit log	Stores an append-only, hash-chained record of security-relevant actions and can verify that the chain is intact.
Web dashboard	Shows threats, charts, system health, detection status, the Response Center, the audit log and settings.
Desktop shell	Starts the Flask server in the background and opens the dashboard inside a native window.

Table 5.1 *Responsibilities of the major components*

5.3 Deployment View

All of SecureX runs on the computer it protects. Nothing is sent to an external server unless the user enables email alerts or an online AI model.

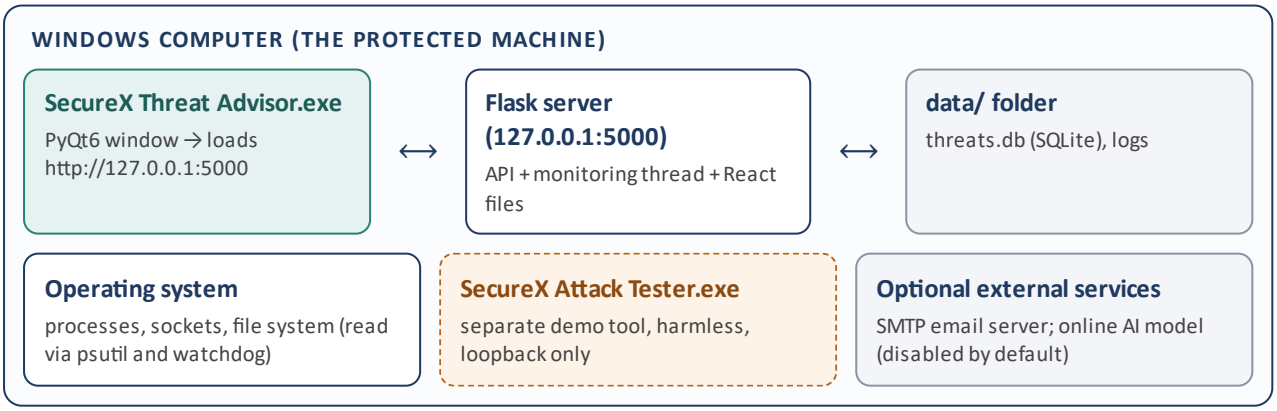


Figure 5.2 *Deployment view on a single Windows computer*

5.4 Data Flow Diagram

Figure 5.3 is the context-level data flow diagram (DFD Level 0). It treats SecureX as a single process and shows the external entities it exchanges data with, and the data stores it uses.

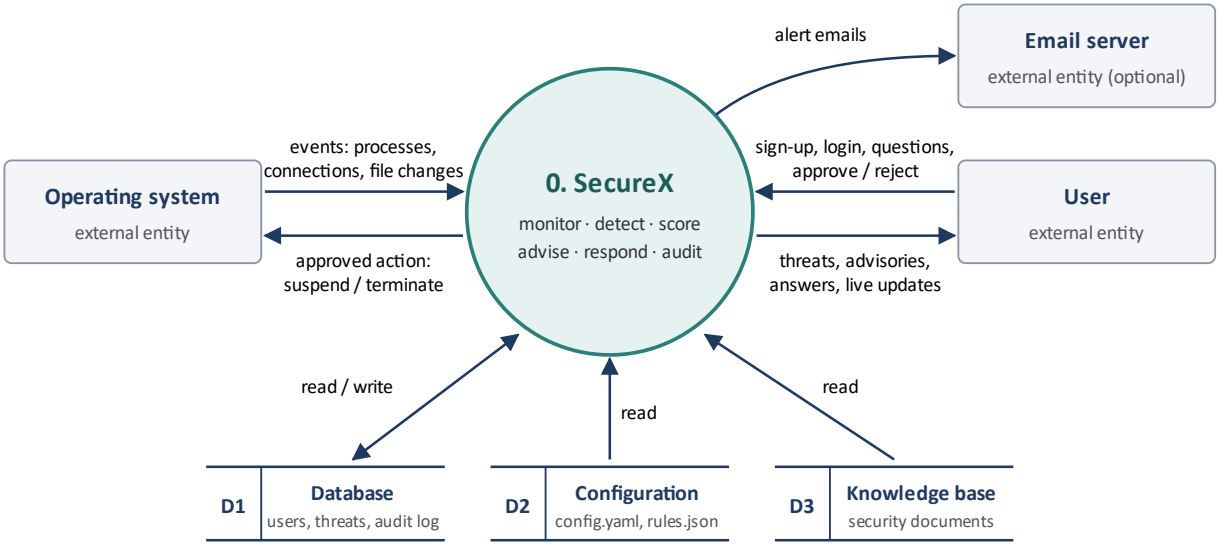


Figure 5.3 *Context-level data flow diagram (DFD Level 0)*

Data flow summary

Event → Threat → Risk → Advisory → Stored threat → (Proposal → Decision) → Audit entry. Monitors produce *events* (for example "process `nmap_sim.exe` started"). The detection engine converts a matching event into a *threat*. The risk scorer adds a score and level, and the advisor adds an explanation. The threat is saved in the database and broadcast to every open dashboard. For serious threats a *proposal* is stored, and the user's decision is written to the audit log.

CHAPTER 6

Low-Level Design (LLD)

The low-level design describes the internal structure of each module: the main files, the order of processing, the formulas and thresholds, the database tables and the API.

6.1 Module Design

Module (file)	Main class / function	Purpose
src/monitors/system_monitor.py	SystemMonitor	Lists processes; detects suspicious names and high CPU or memory use.
src/monitors/network_monitor.py	NetworkMonitor	Lists connections; detects backdoor ports and excessive connections.
src/detection/rule_engine.py	RuleEngine.check_event()	Matches events against the 10 rules in config/rules.json.
src/detection/ransomware_monitor.py	RansomwareMonitor, attribute_process()	Correlates file events over a sliding window; finds the likely responsible process.
src/detection/behaviour_baseline.py	BehaviourBaseline.observe()	Learns normal per-process behaviour with an Isolation Forest and flags outliers.
src/risk_assessment/risk_scorer.py	RiskScorer.calculate_risk()	Weighted risk score, level, and context adjustment.
src/risk_assessment/asset_context.py	AssetContextResolver	Device criticality, network exposure and privileged-account multipliers.
src/advisory/advisor.py	SecurityAdvisor	Plain-language summaries and recommended actions.
src/response/response_manager.py	ResponseManager	Propose, approve, reject and resume containment actions.
src/assistant/ai_assistant.py	AIAssistant	Question answering with retrieval and evidence.
src/notifications/	NotificationManager	Email alert rules, queue and cooldown.
src/audit/audit_log.py	AuditLog.record(), verify_chain()	Hash-chained audit records and integrity check.
src/database/db_manager.py	DatabaseManager	All SQLite reads and writes.

`src/dashboard/app.py`	`perform_scan(), routes`	Flask app, REST API, monitoring loop.
`desktop_app.py`	`main()`	Starts the server and opens the desktop window.
`attacker/`	`simulator.py, attacker_app.py`	Separate, harmless Attack Tester for demonstrations.

Table 6.1 *Module-level design*

6.2 Scan Pipeline

When monitoring is on, the function `perform_scan()` runs repeatedly. Figure 6.1 shows the steps of one cycle.

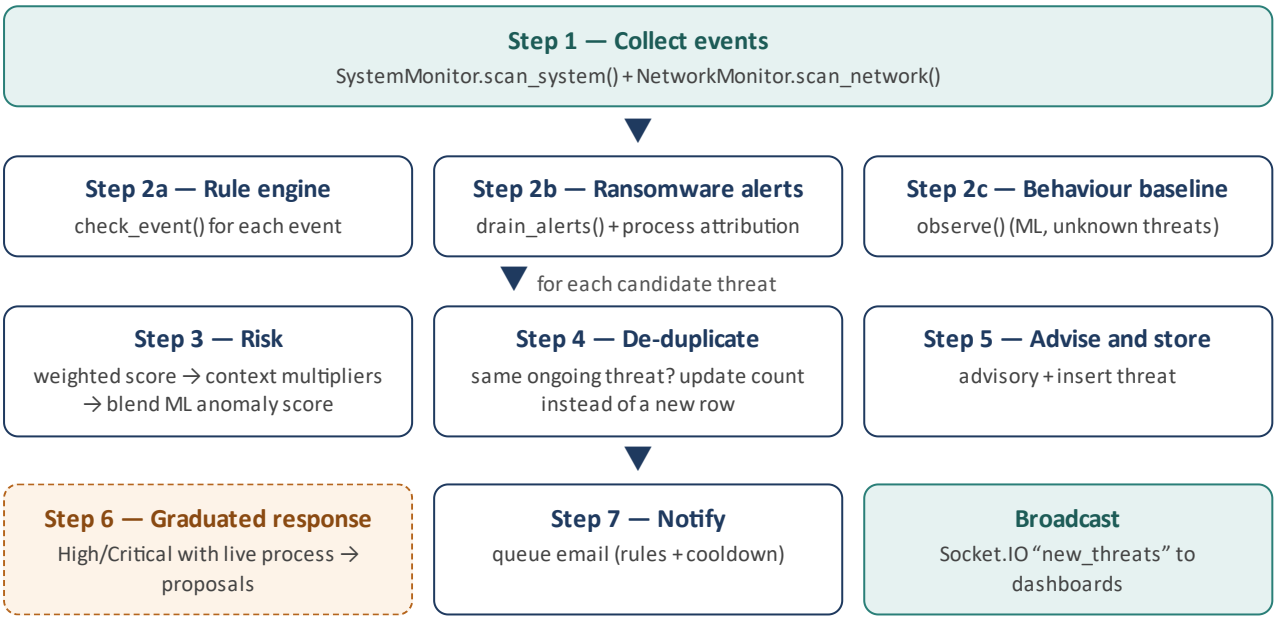


Figure 6.1 *Scan pipeline (one monitoring cycle)*

6.3 Detection Logic

6.3.1 Rule-based detection

ID	Rule	Condition (simplified)	Severity
R1	Suspicious process execution	Process name contains mimikatz, metasploit, nmap, netcat or nc.exe	High
R2	Multiple failed logins	5 or more authentication failures within 60 s	Medium
R3	Unusual port connection	Connection to port 4444, 5555, 6666 or 31337	Medium
R4	Excessive CPU usage	A process above 90% CPU	Low

R5	Critical system file modification	.exe/.dll/.sys changed under System32	High
R6	Excessive network connections	One process with more than 100 connections	Medium
R7	Privilege escalation attempt	Standard user becomes administrator	High
R8	Suspicious script execution	.ps1, .bat, .vbs or .js executed	Medium
R9	Data exfiltration indicator	More than 100 MB sent out within 5 minutes	High
R10	Memory injection	One process injects code into another	High

Table 6.2 *Detection rules (config/rules.json)*

6.3.2 Ransomware behaviour detection

File events in watched folders are kept in a 30-second sliding window. Five signals are checked:

Signal	Triggered when (default threshold)
Bulk modification	150 or more different files created or modified
Rename burst	40 or more files renamed
Extension change	20 or more files renamed to the same unusual extension (e.g. `.locked`)
Ransom note	A file named like a ransom note appears (e.g. `HOW_TO_DECRYPT_FILES.txt`)
Many directories	Activity spread across 15 or more folders

Table 6.3 *Ransomware behaviour signals*

One signal alone gives “Suspicious bulk file modification”. Two signals, an extension change or a ransom note give “Possible ransomware-like behaviour”. Because Windows does not report which program changed a file, SecureX **attributes** the activity in two steps. It keeps a cheap snapshot of each process’s file-operation count every 10 seconds; when an alert arrives, it shortlists the processes that were busy enough to cause the burst and checks which of them has files open in the affected folder. If exactly one matches, it is named as the suspect so that the Response Center can propose containment. This reduced attribution time from about 70 seconds to about 5 seconds.

6.3.3 Unknown-threat detection (machine learning)

During a *learning phase*, SecureX records eight features for every process in each cycle: CPU %, memory %, thread count, connections, distinct remote IPs, distinct remote ports, public remote IPs and suspicious-port connections. An **Isolation Forest** [6] is trained on these normal samples. The alert threshold is set at the 99.5th percentile of the training scores, so only about 0.5% of normal behaviour would score that high. A range check complements the model: a process is also flagged if two or more

features far exceed anything seen during learning. Results are labelled "Unknown suspicious activity", together with the features that deviated most.

6.4 Risk Scoring

Each threat receives a base score between 0 and 1:

$$\text{Risk} = 0.4 \times \text{Severity} + 0.3 \times \text{Confidence} + 0.2 \times \text{Impact} + 0.1 \times \text{Prevalence}$$

The base score is then multiplied by context factors and capped at 1.0:

Context factor	Multiplier
Device criticality (set by the administrator)	low ×0.85 · medium ×1.0 · high ×1.15 · critical ×1.3
Network exposure	local ×1.0 · internal ×1.05 · internet ×1.2
Privileged account (SYSTEM, Administrator)	×1.15
Off-hours activity (10 pm to 6 am)	×1.1
Repeat offender	×1.2

Table 6.4 *Context multipliers*

When a trained ML model is available, its anomaly score is blended in: `blended = 0.7 × score + 0.3 × ML score`, and the higher of the two values is kept. The final score is mapped to a level: **Low** (below 0.3), **Medium** (0.3 to below 0.6), **High** (0.6 to below 0.85) and **Critical** (0.85 and above). Every adjustment is stored with the threat so the user can see why it was rated that way.

6.5 Response State Machine

The policy maps risk level to the strongest allowed response: Low → observe, Medium → alert, High → contain (suspend), Critical → terminate (suspend or terminate). A proposal moves through the states in Figure 6.2.

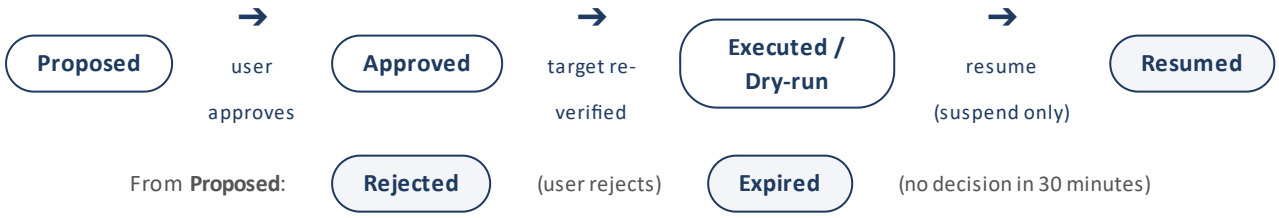


Figure 6.2 *Life cycle of a response proposal*

Safety checks before any action: the process must still be the same one (its ID and start time are compared), system processes such as `lsass.exe` and `explorer.exe` are protected, SecureX never targets itself, at most five actions run per minute, and execution is **off by default**, so approvals are recorded as dry runs until an administrator enables it.

6.6 Database Design

SecureX uses one SQLite database with 17 tables. The core relationships are shown in Figure 6.3 and the most important tables in Table 6.5.



Figure 6.3 *Core database relationships*

Table	Key columns	Purpose
`users`	id, username (unique), email (unique), password_hash, created_at, last_login	Accounts created by sign-up
`threats`	id, threat_name, category, risk_level, risk_score, severity, confidence, detected_at, last_seen_at, occurrence_count, risk_context, evidence, status	Every detected threat
`response_actions`	id, threat_id, action, response_level, target_pid, target_name, target_create_time, status, proposed_at, expires_at, decided_by, result_message	Containment proposals and decisions
`audit_log`	id, created_at, actor, action, target_type, target_id, outcome, details, source_ip, prev_hash, entry_hash	Tamper-evident history
`ai_conversations`	id, user_id, query, response, confidence, created_at	Per-user assistant history

Table 6.5 *Key database tables*

6.7 API Design

Method	Endpoint	Purpose
POST	`/api/auth/signup`	Create an account (validated) and sign in
POST	`/api/auth/login`	Sign in; `/api/auth/logout` signs out
GET	`/api/auth/me`	Current user (401 if not signed in)
POST	`/api/monitoring/start`	Start continuous monitoring (`/stop` to stop)
POST	`/api/scan/start`	Run a single scan
GET	`/api/threats`	List threats; `/api/threat/<id>` for details
GET	`/api/stats, /api/threat-timeline`	Dashboard counters and chart data
GET	`/api/responses`	Response proposals and their outcomes
POST	`/api/responses/<id>/approve`	Approve (also `/reject` and `/resume`)
POST	`/api/assistant/chat`	Ask the AI assistant a question
GET / DELETE	`/api/assistant/history`	Read or delete the user's own chat history
GET	`/api/audit, /api/audit/verify`	View the audit log and verify the hash chain
POST	`/api/phishing/analyze`	Check a URL for phishing signs (never opened)

Table 6.6 *Main REST API endpoints (all except sign-up, login and public routes require sign-in)*

6.8 Security Design

- **Passwords** are stored only as salted hashes (Werkzeug); plain text is never saved. Sign-up enforces username format, a valid email and a minimum password length of 8 characters, and rejects duplicate usernames or emails.
- **Sessions** use Flask's signed cookies with a secret key kept in a `.env` file.
- **Authorisation:** every data and action endpoint requires sign-in, and assistant history is isolated per user.
- **The AI assistant has no tools**: it cannot run commands or stop processes. Retrieved text is passed as untrusted data to resist prompt injection.
- **Audit hash chain:** each entry stores `SHA-256(content + previous hash)`, so editing or deleting any row breaks the chain. Secrets such as passwords are masked before logging.
- **Privacy:** the file monitor records only paths and extensions, never file contents. The phishing checker never opens the URL.

CHAPTER 7

Implementation and User Interface

The backend is written in Python and organised into the packages in Table 6.1. The frontend is a React single-page application built with Vite and served by Flask. The desktop application packages Python, the server and the built frontend into a Windows executable using PyInstaller; when opened, it starts the server on `127.0.0.1:5000` and shows the dashboard in a native window.

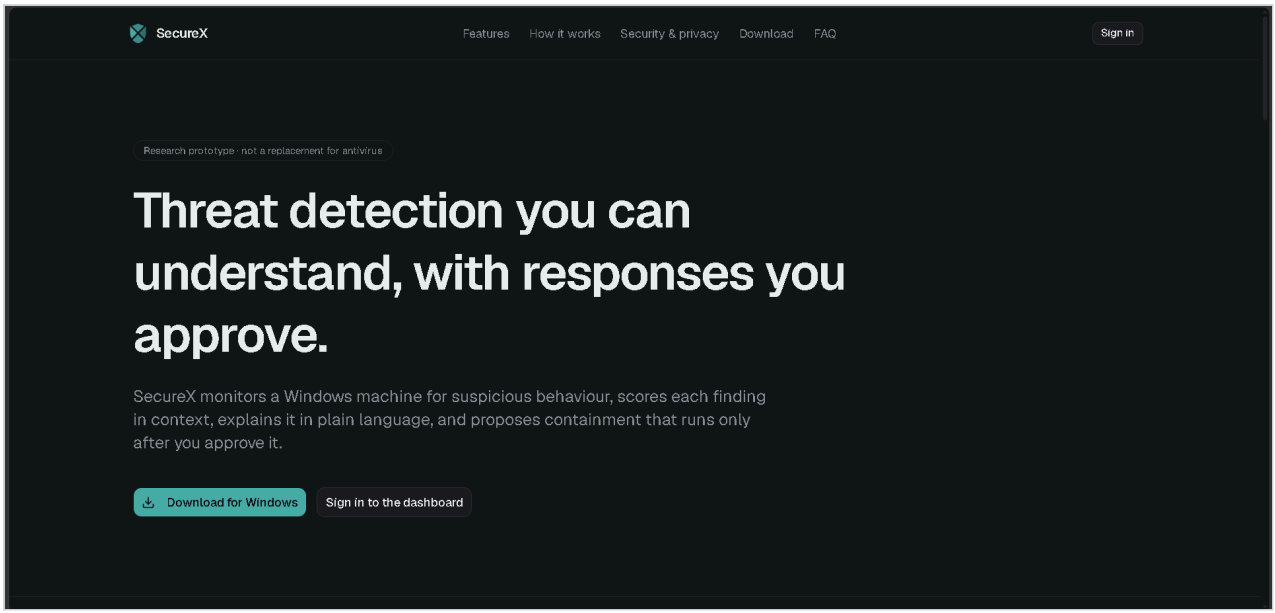


Figure 7.1 *Public landing page with download, sign-in and sign-up options*

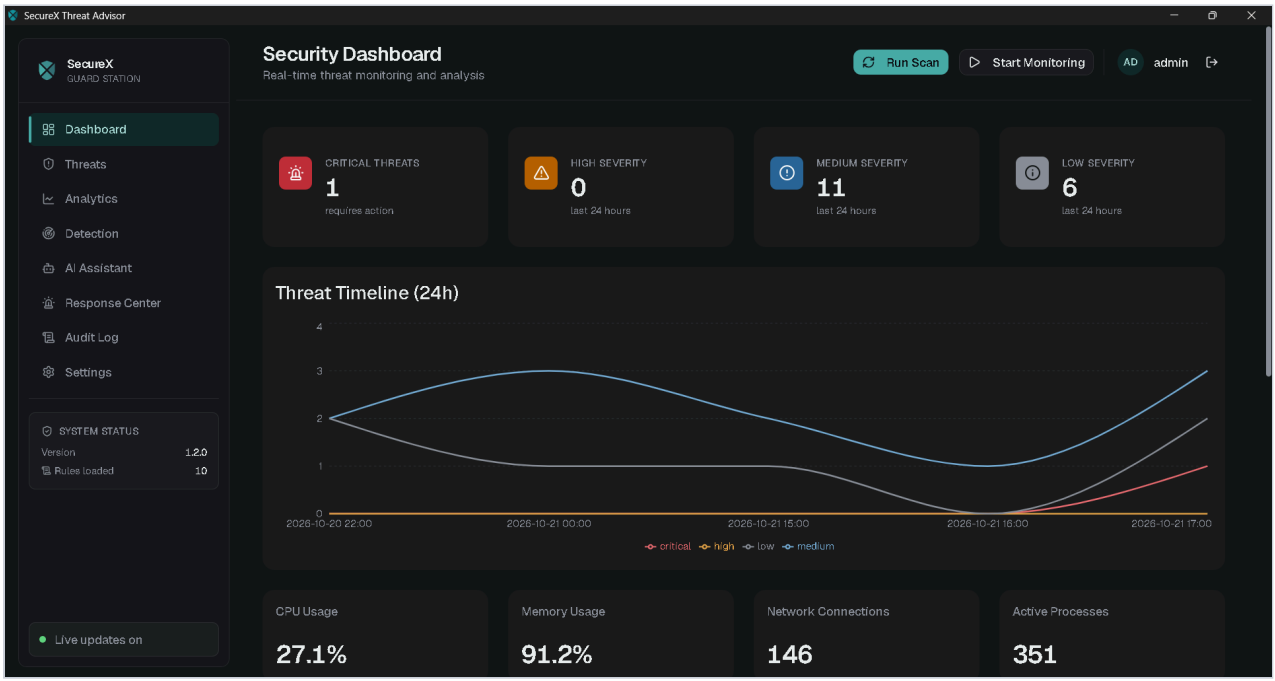


Figure 7.2 *Security dashboard showing severity counts, the 24-hour threat timeline and live system metrics*

7.1 Application Pages

Page	What the user sees and does
Landing, Sign-up, Sign-in	Product overview, account creation and sign-in.
Dashboard	Counts of Critical, High, Medium and Low threats, the 24-hour timeline, and CPU, memory, connection and process counts. Buttons to run a scan or start monitoring.
Threats	Searchable list of threats; a detail view with evidence, risk reasons, advisory and “Explain with AI Assistant”.
Analytics	Charts of threats by category, severity and time.
Detection	Status of the ML baseline and ransomware monitor; phishing URL checker.
AI Assistant	Chat in simple language, with evidence and an option to delete history.
Response Center	Pending proposals with Approve, Reject and Resume buttons.
Audit Log	Full action history with a “verify integrity” check.
Settings	Email alert preferences, system information.

Table 7.1 *Pages of the SecureX dashboard*

CHAPTER 8

Testing and Results

8.1 Automated Testing

The project contains **212 automated test functions in 20 test files**, written with pytest. They cover the rule engine, risk scorer, detectors (ransomware, behaviour baseline, phishing), ML models, database layer, authentication and sign-up, dashboard API, response governance, audit log, AI assistant, notifications and the desktop application. Examples of what is checked:

- Sign-up saves a hashed password, signs the user in, and rejects duplicate or invalid input.
- A burst of renames to `.locked` is classified as possible ransomware, and the alert is attributed to the process holding a file open in that folder.
- Users can see and delete only their own assistant history.
- Protected processes are never proposed for termination, and approvals re-verify the target.
- Editing an audit row is detected by the integrity check.

8.2 Live Demonstration with the Attack Tester

To demonstrate SecureX safely, a separate **Attack Tester** application was built. It imitates attacks without causing harm: network activity stays on the local loopback address (127.0.0.1), dummy files are created in a temporary sandbox folder, and everything is cleaned up afterwards. With monitoring running, all scenarios were executed together, and SecureX produced the results in Table 8.1. This shows that the whole pipeline works end to end; it is **not** a measure of accuracy against real-world malware, which is measured separately with the dataset evaluation pipeline (Chapter 2, gap G1).

Scenario	What the tester does (harmless)	SecureX result	Risk
Process	Runs a renamed copy of `ping.exe` called `nmap_sim.exe`	Suspicious Process Execution	Critical
Backdoor	Opens a loopback connection to port 4444	Unusual Port Connection	High
Port scan	Opens 120 loopback connections from one process	Excessive Network Connections	High
Ransomware	Creates 50 dummy files, renames them to `.locked` and writes a fake ransom note	Possible ransomware-like behaviour; Response Center proposed suspending the responsible process	Critical / High
Brute force	Sends 8 wrong passwords for a fake user	All attempts rejected and recorded in the Audit Log	—

Table 8.1 *Attack Tester scenarios and observed detections (all five detected)*

CHAPTER 9

Limitations and Future Work

9.1 Limitations

- **Prototype scope:** SecureX protects one Windows computer and does not replace antivirus software or scan file contents for signatures.
- **Simple rules:** some rules rely on process names, which an attacker could change. The ML baseline helps, but it assumes the machine is clean while learning.
- **Ransomware attribution is best-effort:** it works when exactly one busy process has files open in the affected folder; otherwise no process is named.
- **Limited response actions:** only suspend, resume and terminate a process. There is no file quarantine, firewall blocking or network isolation.
- **No automatic retraining:** the ML model is retrained manually; concept drift is not detected.
- **Heuristic context multipliers:** the values are transparent but not statistically calibrated.
- **Audit chain:** tampering is evident, but someone with write access to the database could rebuild the chain.

9.2 Future Work

- Kernel-level event sources (for example Windows ETW or Sysmon) to know exactly which process changed each file.
- More response actions: quarantine files, block an IP in the firewall, isolate the network.
- Drift detection with automatic retraining of the behaviour baseline.
- Central management of many computers, with role-based access (administrator and viewer).
- Calibrating the risk multipliers with real incident data, and reporting accuracy on CIC-IDS2017 and UNSW-NB15.
- Code-signing the installer and adding automatic updates.

CHAPTER 10

Conclusion

SecureX shows that a threat detection system can be both technically sound and easy to understand. It combines expert rules, behaviour-based ransomware detection and machine-learning anomaly detection, judges each finding in context with a transparent risk score, and explains the result in plain language. Most importantly, it keeps the human in control: containment actions are only proposed, must be approved, can be undone, and are recorded in a tamper-evident log.

The system is complete and usable. It provides a web dashboard, a Windows desktop application, an AI assistant and user accounts. It is supported by 212 automated tests and a harmless Attack Tester that demonstrates every detection path live. The project addresses all seven identified research gaps in working code, while stating its limits honestly, and provides a solid base for further work toward a production-grade, multi-device security platform.

References

1. A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
2. N. Shone, T. N. Ngoc, V. D. Phai and Q. Shi, "A Deep Learning Approach to Network Intrusion Detection," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 1, pp. 41–50, 2018.
3. Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.
4. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
5. N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," in *Proc. Military Communications and Information Systems Conference (MilCIS)*, 2015.
6. F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE Int. Conf. on Data Mining (ICDM)*, 2008, pp. 413–422.
7. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
8. MITRE Corporation, "MITRE ATT&CK: Adversarial Tactics, Techniques and Common Knowledge," https://attack.mitre.org.